Scion, Mononoke, and what Anthropic replaced Git with at 30,000 merges a day

On 13 October 2026, Anthropic published an engineering account of Scion, an in-house version control system that replaced Git for its primary monorepo after a five-month migration. In May 2026, the post reports, Anthropic averaged roughly 2,500 merged code changes per day; by September 2026, daily merges exceeded 30,000. Simon Willison's 13 October note reproduced the chart with public model-release dates overlaid, cautioning that those dates do not mark when the models became available internally. Two days after publication, following a public critique by former Meta engineer Rain, Anthropic updated the post to credit Meta's open-source Mononoke server and its Bonsai data model.

> In May 2026, Anthropic averaged around 2,500 code changes merged per day; as of September, we merged over 30,000 per day.

Why Anthropic moved off Git

Anthropic's post describes a workload shaped by automated coding agents and continuous-integration jobs rather than offline human laptops. The history of its `main` branch is almost 100 GB, and a compressed working copy exceeds 25 GB. Continuous integration alone runs on the order of 25 million code checkouts per day before counting developer environments, agent sandboxes, and research jobs — a volume that, fetched naively, would exceed 600 PB of daily Git traffic. At its peak, Anthropic operated several hundred internal Git mirrors alongside shallow clones, sparse checkouts, promisor remotes, and repository snapshots ("seeds") in blob storage. As the merge rate climbed, snapshots fell further behind the tip of `main` between refreshes, and keeping history-dependent operations such as `diff` and `blame` working on partial clones became harder to maintain.

Git is decentralized: every full clone can compute diffs, inspect history and blame, and check out revisions offline. Anthropic writes that nearly all of its source-control clients are agents and automated jobs that are online by definition. Asked on Hacker News about the absence of offline support, Scion engineer Nick G. wrote, as quoted by Willison: "Since Claude is by far the heaviest user of Scion and is necessarily online, this just wasn't important to do."

Changesets, derived data, and the three-tier cache

In Anthropic's description, Scion shifts commit creation, merges, rebases, and history queries onto the server. Its unit of work is a "changeset," an entry in a server-side write-ahead log. Where a Git commit stores a pointer to a root tree, a Scion changeset stores the list of files changed relative to its first parent together with content-addressed pointers to compressed file contents in blob storage. Accepting a changeset requires no packfiles, no cross-server locking, and no ref updates during commit creation; moving a ref such as `main` is a separate compare-and-swap on a single Spanner row.

Because a changeset records only modified paths, it cannot by itself answer queries such as who last edited a line or which commits touched a path. To serve those queries over an API without requiring clients to hold a local repository, Scion computes additional server-side structures from changesets — including blame and path-history indexes and skip lists for the commit graph — which the post calls "derived data." Each derived-data type is defined recursively from the values on a changeset's parents and can be computed eagerly or deferred.

For workloads that do materialize files, the `scn` client does not negotiate packfiles by advertising what it already holds. Instead, it requests specific content-addressed trees and blobs recursively through a three-tier hierarchy that speaks a uniform RPC protocol: a node-local `scion-daemon` initialized from periodic snapshots, regional in-memory `scion-cache` servers that stream objects in the daemon's disk format without delta compression, and authoritative `scion-server` instances backed by Spanner and cloud storage. Anthropic reports cache hit rates above 99 percent at each tier, a server CPU and memory footprint 200 times smaller than its former Git mirror fleet, checkouts that finish in seconds rather than minutes, and a 25 percent CPU capacity reduction in CI.

Mononoke, Bonsai, and the 15 October attribution update

On 14 October, Rain — who led the 2018 effort to design and implement the data model for Meta's open-source Mononoke source-control server — published a comparison and a 15 October follow-up arguing that Scion's architecture closely resembles Mononoke, which Anthropic's post did not initially name. Placing Anthropic's text beside 2022 Oxide candidate materials describing Mononoke's Bonsai data model, Rain pointed out that both systems replace Git's root-tree pointer with a write-ahead changeset listing modified files and blob hashes, and that both build secondary recursive Merkle/index structures — called "derived data" in both systems — so operations like `blame` scale with the commits touching a file rather than the entire repository.

Anthropic's original 13 October text credited Google's Piper and Meta's Sapling only for "hard-won lessons." In the 15 October post, Rain wrote that the issue was attribution rather than copying an open-source design, and that a footnote naming Mononoke and its Bonsai changesets would be the appropriate credit; she also described the five-month rebuild as a notable engineering achievement.

On 15 October, Anthropic updated its article to add a footnote stating that Sapling's server component, Mononoke, "pioneered many of the techniques we use in Scion, in particular the Bonsai data model." Replying on Hacker News, Nick G. wrote that the team had initially referred to Sapling because Meta uses that name externally for the whole system, said that ex-Meta employees at Anthropic had their experience of Mononoke as a "north star" from the beginning while adapting decisions to different needs, and cited Martin Kleppmann's 2015 "Turning the database inside-out" as another source for the "derived data" phrasing. In a separate comment, answering how much of Scion's code was directly inspired by Mononoke ("Very little, overall"), he added that Scion began as a read-only service ingesting Git data to offload CI graph queries and ran for months in a hybrid Git/Scion setup during the migration. Rain updated the 15 October post to state that sufficient attribution had been added.

Where Claude accelerated the build, and where engineers said 'no'

Anthropic's write-up reports that a team of seven engineers built Scion, the Antique code-review tool, code search, and a source-control IAM system in five months while daily merges grew tenfold, starting around the internal rollout of Claude Mythos Preview and Claude Tag. In one example from the post, an early internal snapshot of Claude Mythos Preview rewrote a handwritten prototype of the Git-to-Scion ingestion pipeline into a version that used 20 times less memory and ran four times faster while preserving ingestion-ordering invariants.

The post is equally specific about where autonomous coding required human restraint: "Very often, we found that the role of the engineers was to tell Claude 'no.'" During the migration, multiple systems attempted to determine when a change merged to `main` by lin-

early scanning the commit log rather than querying Antique. And while the team used adversarial model review, the post notes that such reviews could be myopic — in one instance prompting Claude to add excessive validation and complexity out of distrust toward another internal service. At the workflow level, Anthropic writes that a global merge-queue lock stopped making sense when hundreds or thousands of changes merged while tests ran, leading the team to shift toward post-merge and pre-deploy automated testing and to rely on Claude to resolve resulting merge conflicts. Anthropic calls Scion "by no means a suitable replacement for git for general use" and says it would be difficult to offer in a broadly useful form, because it depends on bespoke internal code review, code search and CI; it also notes that its internal token spending far exceeds what other businesses would realistically want. Instead, it proposes bringing Scion's server-side ideas to Git forges.

Contributor: Alessandro Linzi